Homework 06

Question 1 Answers

Seed: 43261 (one 70/30 split: 398 training, 171 test, used for Questions 1 and 2). KNN probabilities are the fraction of malignant cases among the k nearest training observations.

```
library(pROC)

bc <- read.csv("data/breast-cancer.csv")
predictors <- c("mean_radius", "mean_texture", "mean_smoothness")

set.seed(43261)
train_id <- sample(nrow(bc), round(0.7 * nrow(bc)))
train <- bc[train_id, ]
test <- bc[-train_id, ]

# Standardize with training means and SDs only.
mu <- colMeans(train[, predictors])
s <- apply(train[, predictors], 2, sd)
X_train <- scale(train[, predictors], center = mu, scale = s)
X_test <- scale(test[, predictors], center = mu, scale = s)
y_train <- train$y
y_test <- test$y

# Euclidean KNN with equal weights; returns P(y = 1) with one column per k.
knn_prob <- function(X_train, y_train, X_new, ks) {
  d2 <- outer(rowSums(X_new^2), rowSums(X_train^2), "+") -
    2 * X_new %*% t(X_train)
  t(apply(d2, 1, function(d) {
    (cumsum(y_train[order(d)]) / seq_along(d))[ks]
  }))
}

get_roc <- function(y, p) roc(y, p, levels = c(0, 1), direction = "<", quiet = TRUE)
```

```
fit <- glm(y ~ ., data = data.frame(y = y_train, X_train), family = binomial())
ks1 <- c(5, 15, 25, 35, 45)
p_test1 <- cbind(
  predict(fit, newdata = data.frame(X_test), type = "response"),
  knn_prob(X_train, y_train, X_test, ks1)
)
models <- c("Logistic", paste0("KNN, k = ", ks1))
rocs1 <- lapply(1:6, function(m) get_roc(y_test, p_test1[, m]))
auc1 <- sapply(rocs1, function(r) as.numeric(auc(r)))
print(data.frame(model = models, test_AUC = round(auc1, 4)))
```

```
        model test_AUC
1    Logistic   0.9888
2  KNN, k = 5   0.9657
3 KNN, k = 15   0.9899
4 KNN, k = 25   0.9906
5 KNN, k = 35   0.9917
```

```
6 KNN, k = 45   0.9927
```

```r
cols <- c("black", "#1F5A94", "#C84A16", "#2E8B57", "#7A4FA0", "#B8860B")
plot(
  c(0, 1), c(0, 1), type = "n", xaxs = "i", yaxs = "i",
  xlab = "False-positive rate", ylab = "True-positive rate"
)
abline(0, 1, lty = 3, col = "grey60")
for (m in 1:6) {
  lines(1 - rocs1[[m]]$specificities, rocs1[[m]]$sensitivities,
        lwd = 2, lty = m, col = cols[m])
}
legend(
  "bottomright", bty = "n", lwd = 2, lty = 1:6, col = cols,
  legend = sprintf("%s (AUC = %.4f)", models, auc1)
)
```

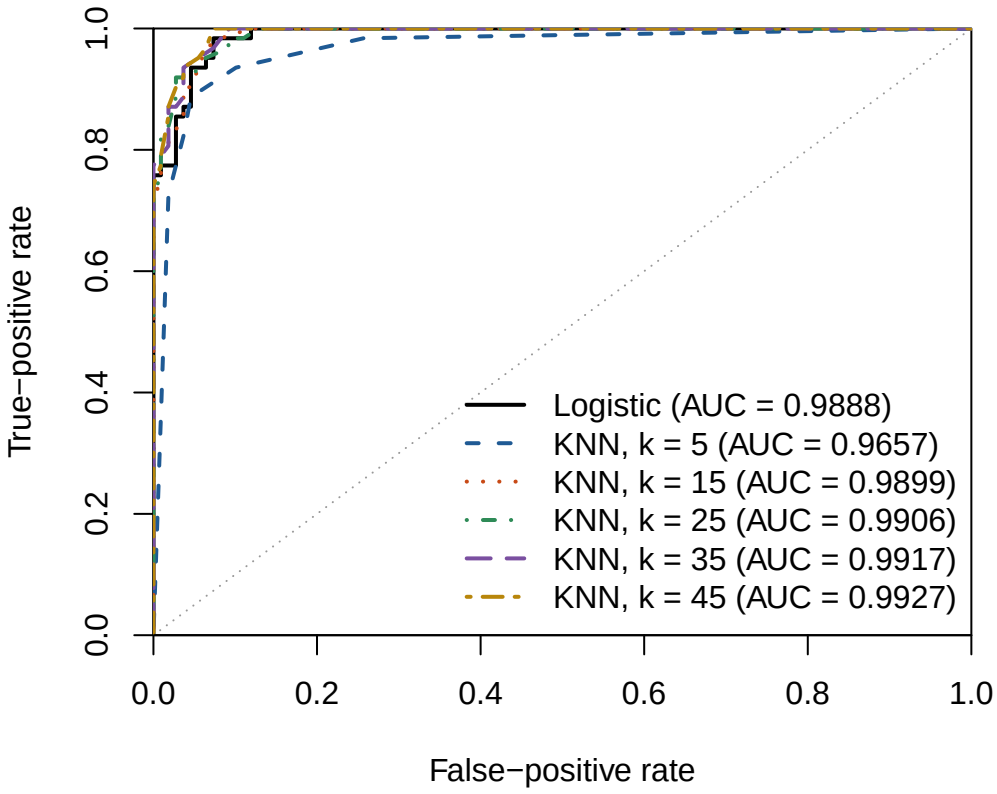


Figure 1: Test ROC curves for logistic regression and KNN.

Test AUCs: logistic 0.9888; KNN 0.9657, 0.9899, 0.9906, 0.9917, 0.9927 for $k = 5, 15, 25, 35, 45$. I prefer logistic regression. KNN with $k = 45$ has the largest AUC, but it beats logistic regression by only 0.004 on one test set of 171 cases, which is within split-to-split noise. Logistic regression gives essentially the same ranking with three interpretable coefficients and no tuning parameter to choose. Only $k = 5$ is clearly worse,

since averaging over 5 neighbors gives noisy probabilities.

Question 2 Answers

```
ks2 <- c(1, 3, 5, 10, 20)
# Using X_train as the new data includes each observation as its own neighbor.
p_train2 <- knn_prob(X_train, y_train, X_train, ks2)
p_test2 <- knn_prob(X_train, y_train, X_test, ks2)
rocs_train <- lapply(seq_along(ks2), function(m) get_roc(y_train, p_train2[, m]))
rocs_test <- lapply(seq_along(ks2), function(m) get_roc(y_test, p_test2[, m]))
auc_train <- sapply(rocs_train, function(r) as.numeric(auc(r)))
auc_test <- sapply(rocs_test, function(r) as.numeric(auc(r)))
print(data.frame(k = ks2, train_AUC = round(auc_train, 4),
                 test_AUC = round(auc_test, 4)))
```

```
   k train_AUC test_AUC
1  1    1.0000   0.9023
2  3    0.9884   0.9496
3  5    0.9824   0.9657
4 10    0.9781   0.9834
5 20    0.9772   0.9908
```

```
par(mfrow = c(2, 3), mar = c(4, 4, 2, 0.5))
for (m in seq_along(ks2)) {
  plot(
    1 - rocs_train[[m]]$specificities, rocs_train[[m]]$sensitivities,
    type = "l", lwd = 2, col = "#1F5A94", xlim = c(0, 1), ylim = c(0, 1),
    xaxs = "i", yaxs = "i", main = paste("k =", ks2[m]),
    xlab = "False-positive rate", ylab = "True-positive rate"
  )
  lines(1 - rocs_test[[m]]$specificities, rocs_test[[m]]$sensitivities,
        lwd = 2, lty = 2, col = "#C84A16")
  abline(0, 1, lty = 3, col = "grey60")
  legend(
    "bottomright", bty = "n", lwd = 2, lty = 1:2, cex = 0.8,
    col = c("#1F5A94", "#C84A16"),
    legend = sprintf(c("Train AUC = %.4f", "Test AUC = %.4f"),
                     c(auc_train[m], auc_test[m]))
  )
}
```

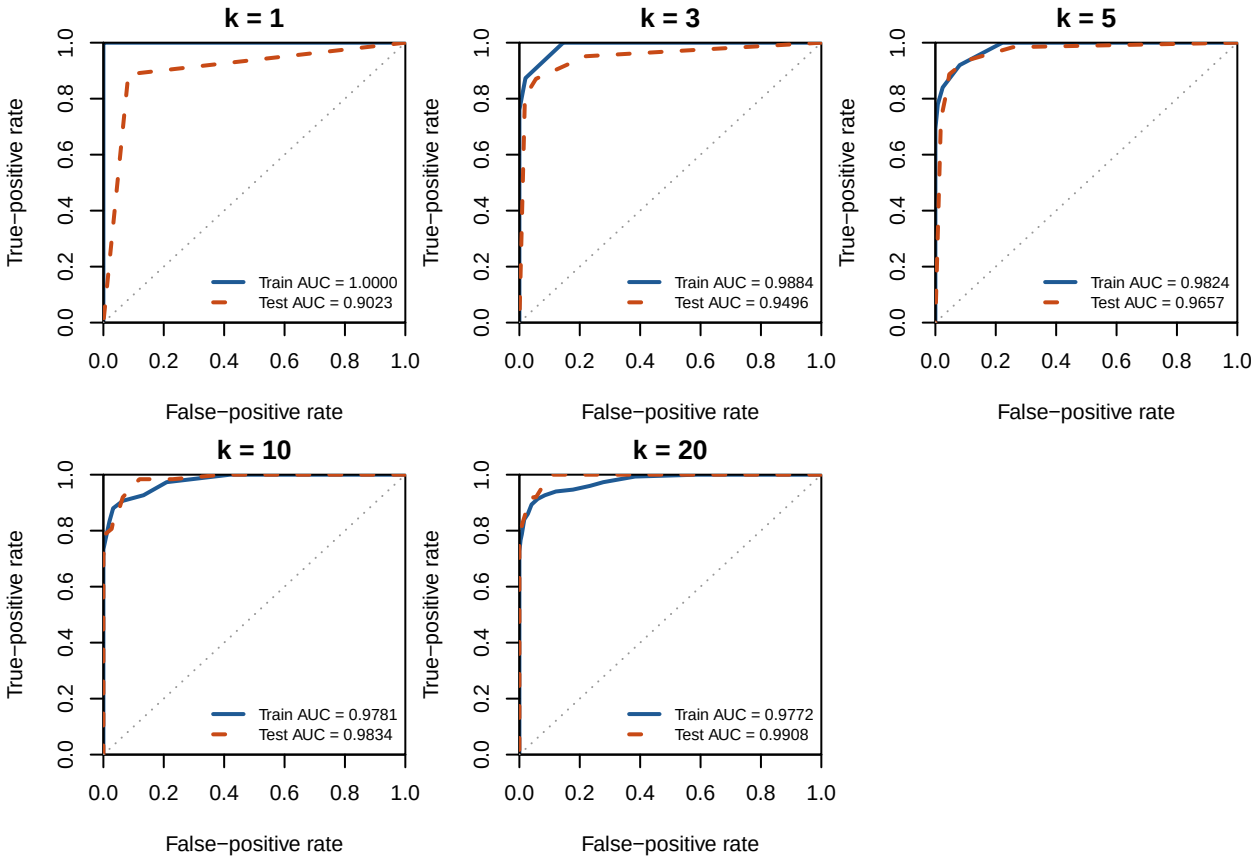


Figure 2: Training (solid) and test (dashed) ROC curves for KNN.

Training AUC falls as k grows while test AUC rises, and the gap shrinks from 0.098 at $k = 1$ to a slightly negative -0.014 at $k = 20$. At $k = 1$, each training point is its own nearest neighbor, so its fitted probability is exactly its observed label and training AUC is a perfect 1. The model has memorized the training labels, including noisy cases near the class boundary, so its test AUC is the lowest of the five. Its probabilities are only 0 or 1, so the test ROC curve has a single corner.

Larger k averages over more neighbors, so a point's own label carries weight $1/k$ and the fit is smoother and less variable. Training AUC measures how well the model reproduces labels it already used, so it is optimistic for small k and does not reflect performance on new observations; only the held-out test set shows that.

Question 3 Answers

Link to posted GitHub Issue: https://github.com/manushrip06/stat432-fall2026/issues/2 ## Part 1

I selected Question 1 Part b from manushrip06's report. The code and plot are correct, but the written explanation was put in Python comments inside the code chunk instead of in the text, so every line runs past the page margin and gets cut off in the PDF, which makes the reasoning harder to follow. Beyond the formatting, the explanation could improve my using simulation results to support it.

Part 2

Quoted from the report (the lines are cut off at the page margin):

```
#An increase in k leads to an average increase
#in number of responses used for training, thereby leading to a decrease in
#However, an increase in k means that we use data further from x_0,
#which ncreasing bias. The MSE decreases initially and then increases due to
```

As we can see, the explanation is missing words that got cut out from the comments in the code block. Because of this, we don't know what decreases as k grows, or what makes MSE increase. A better approach for this would be to keep code and small comments in code blocks, but to use markdown and actual text for explanations as well as leverage LaTeX to accurately show notation. Additionally, the answer could be more specfic with observed results rather than explain the typical bias-variance trade off for KNN.

Part 3

I reran the author's code unchanged (`np.random.seed(1)`, same draws as Part a) and added a summary table and identity check after the simulation loop:

```
summary = pd.DataFrame({"k": k_values, "squared_bias": squared_bias,
                        "variance": variance, "mse": mse})
print(summary.round(5).to_string(index=False))
print("max |MSE - (bias^2 + variance)|:",
      np.max(np.abs(mse - (squared_bias + variance))))
print("k minimizing MSE:", k_values[np.argmin(mse)])
```

k	1	5	9	13	**15**	17	21	25	29
Squared bias	0.00030	0.00264	0.00448	0.00655	**0.00688**	0.00747	0.01003	0.01318	0.01628
Variance	0.03561	0.01588	0.01227	0.00810	**0.00757**	0.00740	0.00699	0.00619	0.00569
MSE	0.03591	0.01853	0.01675	0.01465	**0.01445**	0.01487	0.01702	0.01937	0.02197

Corrected written answer (as markdown text, not code comments):

The identity MSE = bias^2 + variance holds to machine precision ($\approx 7 \times 10^{-17}$). Squared bias is nearly 0 at $k = 1$ (0.0003) and grows steadily to 0.0163 at $k = 29$, because larger neighborhoods reach points where f differs from $f(\mathbf{x}_0)$. Variance falls from 0.0356 at $k = 1$ to 0.0057 at $k = 29$, because the estimate averages more independent responses. MSE is dominated by variance for small k and by bias for large k, so it is U-shaped with its minimum at $k = 15$ (0.0145), where the two components are about equal. Like ridge and lasso, small k corresponds to a weak penalty (flexible, high variance) and large k to a strong penalty (smooth, high bias).